

# Spring Boot 2

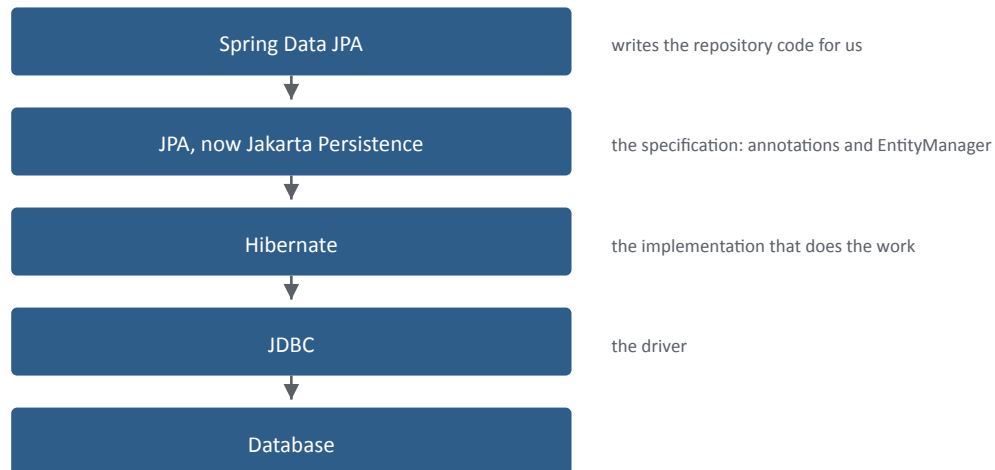
The first Spring Boot document covered the container, dependency injection and auto-configuration. This one covers how an application reaches a database: Spring Data JPA on top, Hibernate underneath.

## Contents

1. [The three layers](#)
2. [Entity mapping](#)
3. [Relationships](#)
4. [Repositories](#)
5. [Fetching and the N+1 problem](#)
6. [The persistence context](#)
7. [@Transactional and transaction management](#)

## 1. The three layers

Three names get used as though they were one thing. They are not.



JPA defines no behavior of its own. Every query is finally run by Hibernate through JDBC.

| Name            | What it is                                                         |
|-----------------|--------------------------------------------------------------------|
| JPA             | A specification. Interfaces and annotations, and no implementation |
| Hibernate       | An implementation of JPA. It generates and runs the SQL            |
| Spring Data JPA | A layer above JPA that writes repository implementations for us    |

In Spring Boot 3 the package changed from `javax.persistence` to `jakarta.persistence`. The annotations are otherwise the same, which is why older examples still read correctly.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

## 2. Entity mapping

An entity is a class mapped to a table.

```
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String name;

    @Column(precision = 10, scale = 2)
    private BigDecimal salary;

    @Enumerated(EnumType.STRING)
    private Status status;

    @Column(name = "hired_on")
    private LocalDate hiredOn;

    protected Employee() { }    // JPA needs a constructor with no arguments

    // getters and setters
}
```

| Annotation      | What it does                                   |
|-----------------|------------------------------------------------|
| @Entity         | Marks the class as mapped to a table           |
| @Table          | Names the table when it differs from the class |
| @Id             | The primary key                                |
| @GeneratedValue | How the key is produced                        |
| @Column         | Names and constrains the column                |
| @Enumerated     | How an enum is stored                          |
| @Transient      | A field that is not stored                     |

The key generation strategy matters more than it looks:

| Strategy | How it works                                                     |
|----------|------------------------------------------------------------------|
| IDENTITY | The database assigns it. Simple, and it prevents batched inserts |
| SEQUENCE | A database sequence. The better choice on PostgreSQL and Oracle  |
| AUTO     | The provider decides                                             |
| TABLE    | A separate table used as a counter. Slow                         |

Store enums as **STRING**, never as **ORDINAL**. Ordinal saves the position in the enum, so inserting a new constant in the middle changes what every existing row means, and nothing fails while it happens.

An entity also needs to not be **final**, and to have a constructor with no arguments, because Hibernate creates instances by reflection and subclasses them for lazy loading.

### 3. Relationships

```
@Entity
public class Department {

    @Id @GeneratedValue
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval =
true)
    private List<Employee> employees = new ArrayList<>();
}
```

```
@Entity
public class Employee {

    @Id @GeneratedValue
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}
```

| Annotation | Meaning                                                 |
|------------|---------------------------------------------------------|
| @ManyToOne | Many rows point at one. This side holds the foreign key |

| Annotation  | Meaning                                             |
|-------------|-----------------------------------------------------|
| @OneToMany  | The other side of that. Needs <code>mappedBy</code> |
| @OneToOne   | One row to one row                                  |
| @ManyToMany | Needs a join table                                  |

The owning side is the one with the foreign key, and only the owning side is written. `mappedBy` says "the other side owns this". Setting only the collection changes nothing in the database:

```
department.getEmployees().add(employee);    // changes nothing on its own
employee.setDepartment(department);         // this is the one that is saved
```

The usual answer is a helper that sets both, so the object graph and the database agree:

```
public void addEmployee(Employee employee) {
    employees.add(employee);
    employee.setDepartment(this);
}
```

`cascade` decides which operations travel to the other side, and `orphanRemoval = true` deletes a child that is taken out of the collection.

**@ManyToMany is usually worth replacing.** A join table almost always gains a column later, such as the date the link was made or the role it represents, and a plain `@ManyToMany` has nowhere to put it. Modeling the join as its own entity with two `@ManyToOne` sides avoids that.

## 4. Repositories

A repository is an interface. Spring Data writes the implementation at startup.

```

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    List<Employee> findByDepartmentName(String name);

    List<Employee> findBySalaryGreaterThanOrderBySalaryDesc(BigDecimal salary);

    Optional<Employee> findByEmail(String email);

    long countByStatus(Status status);

    @Query("select e from Employee e join fetch e.department where e.salary > :min")
    List<Employee> findWithDepartment(@Param("min") BigDecimal min);

    @Modifying
    @Query("update Employee e set e.status = :status where e.id = :id")
    int updateStatus(@Param("id") Long id, @Param("status") Status status);
}

```

Nothing else is written. The method names are parsed into queries.

| Interface                  | What it adds                           |
|----------------------------|----------------------------------------|
| CrudRepository             | save, findById, delete, count          |
| PagingAndSortingRepository | Paging and sorting                     |
| JpaRepository              | flush, batch operations, JPA specifics |

The words a method name can be built from:

| Keyword                        | Produces            |
|--------------------------------|---------------------|
| findBy, readBy, getBy          | A select            |
| And, Or                        | Combined conditions |
| GreaterThan, LessThan, Between | Comparisons         |
| Like, StartingWith, Containing | Patterns            |
| OrderBy...Asc, OrderBy...Desc  | Sorting             |
| First, Top10                   | A limit             |
| countBy, existsBy, deleteBy    | Other verbs         |

Paging is a parameter rather than a keyword:

```

Page<Employee> page = repository.findByDepartmentName(
    "Engineering",
    PageRequest.of(0, 20, Sort.by("salary").descending()));

```

A derived name stops being useful somewhere around three conditions. After that `@Query` is easier to read than `findByDepartmentNameAndStatusAndSalaryGreaterThanOrderByHiredOnDesc`.

## 5. Fetching and the N+1 problem

Every association has a fetch type, and the defaults are not what most applications want:

| Association                                        | Default |
|----------------------------------------------------|---------|
| <code>@ManyToOne</code> , <code>@OneToOne</code>   | EAGER   |
| <code>@OneToMany</code> , <code>@ManyToMany</code> | LAZY    |

Setting everything to `LAZY` and fetching what each query needs is the usual approach, because an eager association is loaded on every query whether it is used or not.

**N+1 is the problem this topic is known for.**

```
List<Employee> employees = repository.findAll();           // one query

for (Employee employee : employees) {
    System.out.println(employee.getDepartment().getName()); // one query each
}
```

With a hundred employees that is a hundred and one queries:

```
select * from employees;           -- 1
select * from departments where id = 1; -- 2
select * from departments where id = 2; -- 3
...
```

Three ways to fix it:

```
// 1. join fetch, in the query itself
@Query("select e from Employee e join fetch e.department")
List<Employee> findAllWithDepartment();

// 2. an entity graph, which keeps the derived method name
@EntityGraph(attributePaths = "department")
List<Employee> findAll();
```

```
// 3. a batch size, which turns 100 queries into 4
@BatchSize(size = 25)
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

**LazyInitializationException** is the same subject from the other side.

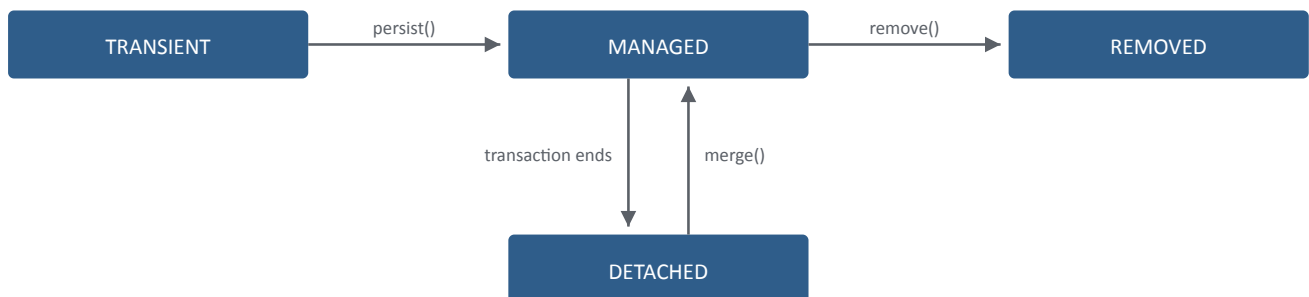
```
org.hibernate.LazyInitializationException:
    could not initialize proxy - no Session
```

It means something read a lazy association after the persistence context had closed. The fix is to load it inside the transaction, with a `join fetch` or an entity graph, or to build a DTO while the transaction is still open.

Spring Boot sets `spring.jpa.open-in-view=true` by default, which keeps the persistence context open for the whole web request and makes the exception disappear. It also holds a database connection for the whole request and moves queries into the view layer, so the usual advice is to switch it off and fix the queries instead.

## 6. The persistence context

The persistence context holds the entities the current transaction is managing. `EntityManager` owns it, and every entity is in one of four states.



Only a managed entity is watched for changes. A detached one can be changed freely and nothing is written.

| State     | Meaning                                          |
|-----------|--------------------------------------------------|
| Transient | A new object. The context knows nothing about it |
| Managed   | Tracked. Changes to it will be written           |
| Detached  | It was managed, and the context has closed       |
| Removed   | Marked for deletion at the next flush            |

**Dirty checking is the behavior that surprises people.** A managed entity that changes is written without anyone calling `save` :

```
@Transactional
public void giveRaise(Long id) {
    Employee employee = repository.findById(id).orElseThrow();
    employee.setSalary(new BigDecimal("80000"));
    // no save() call, and the update still happens
}
```

At the end of the transaction Hibernate compares each managed entity with the snapshot it took when loading it, and writes an `update` for what differs.

The context is also a cache for the length of the transaction. Calling `findById` twice with the same id runs one query, and the second call returns the same object.

`flush` and `commit` are not the same. `flush` sends the SQL to the database. `commit` ends the transaction and makes it permanent.

## 7. @Transactional and transaction management

A transaction groups several writes so that all of them are kept or none of them are.

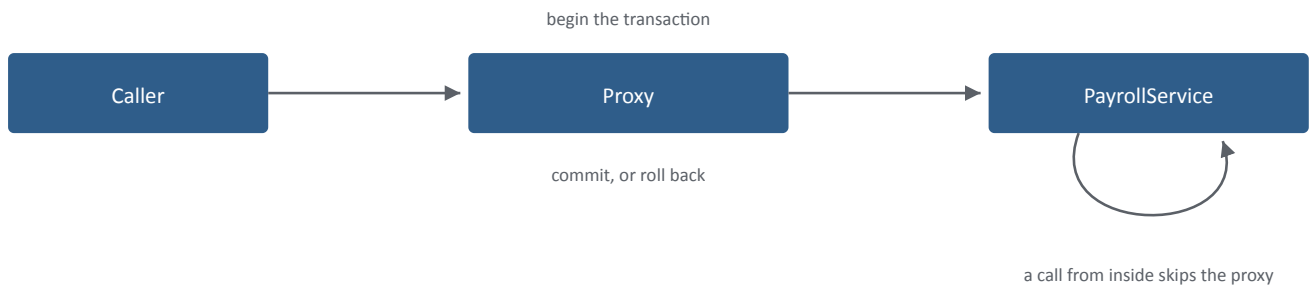
```
@Service
public class PayrollService {

    @Transactional
    public void applyRaise(Long id, BigDecimal amount) {
        Employee employee = repository.findById(id).orElseThrow();
        employee.setSalary(employee.getSalary().add(amount));
        auditRepository.save(new AuditEntry(id, amount));
    }
}
```

If the audit row fails to save, the raise is undone too.

### How Spring applies it

Nothing in that method opens a transaction. Spring wraps the bean in a proxy, and the proxy does the work around the call.



Everything `@Transactional` does happens in the proxy, which is why a call that never leaves the bean gets no transaction.

That single fact explains most of the surprises further down.

## Propagation

`propagation` says what to do when a transaction is already running.

| Value                      | Behavior                                                   |
|----------------------------|------------------------------------------------------------|
| <code>REQUIRED</code>      | Join the running one, or start one. The default            |
| <code>REQUIRES_NEW</code>  | Always start a new one, suspending any running transaction |
| <code>SUPPORTS</code>      | Join one if it exists, otherwise run with none             |
| <code>NOT_SUPPORTED</code> | Suspend any transaction and run without one                |
| <code>MANDATORY</code>     | Join one, and fail if there is none                        |
| <code>NEVER</code>         | Fail if a transaction is running                           |
| <code>NESTED</code>        | A savepoint inside the running transaction                 |

Two of them do most of the work.

`REQUIRED` means a method called from inside another transactional method joins it, so a failure anywhere undoes everything:

```

@Transactional
public void applyRaise(Long id, BigDecimal amount) {
    updateSalary(id, amount); // joins this transaction
    writeAudit(id, amount);   // joins it too
}
  
```

`REQUIRES_NEW` is how a record survives the rollback of the work it describes:

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void recordAttempt(Long id, String outcome) {
    attemptRepository.save(new Attempt(id, outcome));
}
```

The outer transaction is suspended while this one runs and commits on its own. Two transactions are open at once, so two connections are held, which is why this is used sparingly.

`NESTED` uses a savepoint rather than a second transaction. The inner part can be rolled back on its own while the outer one carries on.

## Isolation

`isolation` sets the database isolation level for this transaction. The levels are the database's, not Spring's.

| Value                         | Prevents                                |
|-------------------------------|-----------------------------------------|
| <code>DEFAULT</code>          | Whatever the database is configured for |
| <code>READ_UNCOMMITTED</code> | Nothing                                 |
| <code>READ_COMMITTED</code>   | Dirty reads                             |
| <code>REPEATABLE_READ</code>  | Dirty and non-repeatable reads          |
| <code>SERIALIZABLE</code>     | All three, including phantom reads      |

`DEFAULT` is almost always right. Raising the level for one method is worth doing when that method reads a value and writes a decision based on it.

## Rollback rules

**A rollback happens on unchecked exceptions only.** A checked exception commits the work, which is rarely what anyone wants:

```
@Transactional
public void applyRaise(Long id) throws IOException {
    repository.save(...);
    writeToFile();           // throws IOException, and the save is kept
}
```

Two attributes change it:

```
@Transactional(rollbackFor = Exception.class)           // roll back on anything
@Transactional(noRollbackFor = NotFoundException.class)
```

**Catching the exception cancels the rollback**, because the proxy never sees it:

```
@Transactional
public void applyRaise(Long id) {
    try {
        repository.save(...);
        riskyStep();
    } catch (Exception e) {
        log.error("failed", e);    // the transaction commits
    }
}
```

If the work should be undone, the exception has to leave the method, or the code has to mark the transaction itself:

```
TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();
```

## readOnly

```
@Transactional(readOnly = true)
public List<Employee> findAll() { ... }
```

It does two useful things. Hibernate skips dirty checking, so it takes no snapshots and compares nothing at the end, which is a real saving on a query returning many rows. And the driver can route the call to a read replica where one is configured.

It is a hint, not a guarantee. Some databases reject writes inside it and others allow them.

## The traps

**Self-invocation gets no transaction.**

```
public void outer() {
    inner();    // the call never leaves the object
}

@Transactional
public void inner() { }
```

The proxy wraps the bean from outside. A call from `outer` to `inner` goes straight to the method. Moving `inner` into another bean fixes it, and injecting the bean into itself also works but is harder to read.

`private` and `final` methods get no transaction either, for the same reason: a proxy cannot intercept a method it cannot override.

**A transaction is not the place for a remote call.**

```
@Transactional
public void placeOrder(Order order) {
    orderRepository.save(order);
    paymentClient.charge(order);    // holds the connection for the whole call
}
```

The database connection is held for as long as the method runs. A slow payment provider now consumes a connection from the pool per order, and the pool is usually smaller than the thread count. Save first, commit, and make the remote call after.

## When the annotation does not fit

`TransactionTemplate` does the same job in code, which suits a transaction that covers part of a method:

```
@Service
public class PayrollService {

    private final TransactionTemplate transactionTemplate;

    public void applyRaise(Long id, BigDecimal amount) {
        BigDecimal rate = rateClient.currentRate();    // outside

        transactionTemplate.execute(status -> {
            Employee employee = repository.findById(id).orElseThrow();
            employee.setSalary(employee.getSalary().add(amount));
            return null;
        });
    }
}
```

The remote call sits outside the transaction, which is what the previous trap asks for.

`@Transactional` → the whole method, applied by a proxy  
`TransactionTemplate` → an exact block, applied by us

## The pieces together

| Piece                | What it does            |
|----------------------|-------------------------|
| <code>@Entity</code> | Maps a class to a table |

| Piece                                                 | What it does                                             |
|-------------------------------------------------------|----------------------------------------------------------|
| <code>@ManyToOne</code> with <code>@JoinColumn</code> | The owning side of a relationship                        |
| <code>JpaRepository</code>                            | A repository written for us from the interface           |
| <code>join fetch</code> , <code>@EntityGraph</code>   | Loading an association in the same query                 |
| Persistence context                                   | Tracks managed entities and writes their changes         |
| <code>@Transactional</code>                           | Several writes kept as one unit, applied by a proxy      |
| <code>REQUIRES_NEW</code>                             | A transaction that commits even if the caller rolls back |
| <code>readOnly = true</code>                          | No dirty checking, and a read replica where there is one |

For example, imagine a payroll application. `Employee` and `Department` are entities, and `Employee` owns the foreign key. The repository is an interface with four derived methods and one `@Query` that uses `join fetch`, because the report reads both and would otherwise run a query per row. Giving a raise is a `@Transactional` method that loads the employee, changes the salary and writes an audit row, with no `save` call for the first because dirty checking handles it. `open-in-view` is off, so a missed fetch fails in a test rather than in production.

The most useful thing at this point is not to memorize the annotations. Switch on `spring.jpa.show-sql`, run one screen of a real application, and count the queries.